석 건 | 경력기술서

1. Karpenter 스팟 인스턴스 재생성 이슈

이슈 사항

EKS 환경에서 Karpenter를 통해 스팟 인스턴스를 운영하는 과정에서, 수백 회에 달하는 스팟 인스턴스 재생성 이슈가 발생함

스팟 인스턴스가 생성되자마자 Rebalance Recommendation 이벤트를 수신해 즉시 회수되고, 이후 새로 생성된 인스턴스에서도 동일 이벤트가 반복되는 상황이 지속됨

이 과정에서 스팟 인스턴스에서 구동 중이던 Pod가 다른 노드로 정상적으로 Rollout 되지 못한 채, 노드가 강제 회수되어 서비스에 다운타임이 발생하는 문제가 발생함

따라서 다음 두 가지 원인 확인이 필요

1. Rebalance Recommendation 이벤트가 반복 발생하는 원인
2. Pod가 Rollout 되지 않은 상태에서 노드가 회수되는 원인

문제점

Pod가 Rollout 되지 않은 상태에서 노드가 강제로 회수되는 원인을 확인하기 위해 Karpenter 컨트롤러의 노드 회수 로직 확인

1. Karpenter가 node-termination-handler로부터 스팟 인스턴스 회수 이벤트를 수신 시, `SpotInterruptionKind` 메시지 생성
2. `SpotInterruptionKind` 메세지를 처리하기 위해 `handleMessage()` → `handleNode()` → `deleteNode()` 순서로 메소드를 호출
3. Reconcile 및 Finalize 단계에서, 노드에 `karpenter.sh/termination` finalizer가 존재하면 Cordon 및 Drain 수행
4. Drain 과정에서 Pod가 다른 노드로 Rollout 되지 않고 Eviction 처리되어 결과적으로 서비스 트래픽 해소 불가 및 다운타임 발생

Rebalance Recommendation 이벤트 반복 발생 원인 확인

- Karpenter의 스팟 인스턴스 생성 전략이 기본적으로 `price-capacity-optimized` 로 설정되어 있음
- AWS의 스팟 인스턴스 가용 상황에 따라 스팟 인스턴스가 지속적으로 회수 및 재생성되는 구조
- 외부 환경(AWS 가용성)에 따라 반복 발생하는 구조적 한계임을 확인

개선 사항

- Karpenter 컨트롤러 로직 수정
  - 노드 회수 시, 기존 Drain 방식 대신 Pod를 다른 노드로 Rollout 방식으로 재배치하도록 개선
- 운영 환경 정책 변경
  - 운영 환경에서는 스팟 인스턴스 대신 온디맨드 인스턴스만 사용해 가용성 확보
  - 개발 환경에서는 스팟과 온디맨드 인스턴스를 조합하여 비용 효율화 및 안정성 균형 유지

적용 결과

- 스팟 인스턴스 회수 시에도 Pod가 정상적으로 다른 노드로 이동해 서비스 다운타임 발생 문제 해결
- 운영 환경에서는 스팟 인스턴스 사용을 배제하고 온디맨드 인스턴스만 활용해 가용성 이슈 재발 방지
- 개발 환경에서는 비용 최적화와 안정성을 모두 확보하고, 환경별 특성에 맞춘 안정적 인프라 운영 체계를 구축

2. EC2 to EKS 무중단 마이그레이션

이슈 사항

기존 EC2 환경에서 다음과 같은 문제로 인해 EKS 환경으로의 마이그레이션 필요성이 제기됨

- 비효율적인 자원 관리
  - MSA 아키텍처 적용을 위해 각 API를 별도 EC2 인스턴스로 구성해 독립성은 확보했지만, 자원 활용 측면에서 비효율 발생

- 배포 시간 지연
  - 사내 배포 시스템의 제약으로 인스턴스 프로비저닝, 서비스 구성, ELB Health Check까지 약 10분 소요
  - 신규 서비스 배포 시 Packer 기반 이미지 생성으로 추가 15분 소요
- 다양한 배포 방식 지원
  - EC2 환경에서는 사내 배포 시스템 제약으로 Blue-Green 배포만 가능
  - EKS 환경에서는 Argo Rollouts 도입으로 카나리 배포 전략 활용 가능
    - Argo Rollouts 권한 관리 부재 문제는 ArgoCD Extension을 통해 애플리케이션별 권한 제어로 해결

의사 결정 사항

- 브랜치 전략
  - Gitflow Workflow
  - Trunk Based Development
- 빌드 및 배포 전략
  - Jenkins 기반
    - 빌드: Jenkins → CodeBuild
    - 배포: Jenkins → ArgoCD
    - 통합: Jenkins → CodeBuild → ArgoCD
  - GitHub Webhook 기반
    - 빌드 및 배포: GitHub Webhook(Push/Merge) → CodeBuild → ArgoCD

실행 방안

방안 1. Blue-Green 방식 - 도메인 소유권 이전

1. EKS 환경에 ALB → Ingress → Ingress Gateway → Gateway → VirtualService → Service → Pod 구조로 리소스 구성
2. Gateway 리소스에서 External-DNS Annotation을 통해 서비스 도메인 생성
3. 기존 도메인 소유권 이전을 위해 Route53에 TXT 레코드 추가

`"heritage=external-dns,external-dns/owner=(클러스터명),external-dns/resource=gateway/(네임스페이스)/(Gateway 리소스명)"`

4. External-DNS가 1분 주기로 도메인 상태를 동기화, 서비스 도메인이 자동으로 EKS ALB로 전환
5. EC2 기반 ALB에 트래픽 유입이 없는 경우 ALB 삭제

방안 2. 가중치 기반 점진적 트래픽 이관

1. EC2와 EKS 도메인에 각각 가중치 설정
2. EC2 도메인 가중치를 점진적으로 낮춰 트래픽을 EKS로 이관
3. EC2 ALB로의 트래픽 유입 종료 시 EC2 도메인 삭제
4. EKS 도메인의 가중치 설정 해제

적용 결과

- 배포 시간 10분 → 2분으로 단축
- EC2 인스턴스에서 컨테이너 기반 아키텍처로 전환, 자원 관리 효율성 개선
- 빠른 배포와 유연한 스케일링 대응으로 서비스 가용성 향상
- 무중단으로 EC2 → EKS 마이그레이션 완료

3. Netty 애플리케이션에서 발생한 CoreDNS 리졸빙 장애 원인 분석

이슈 사항

EKS 클러스터를 1.29에서 1.30으로 업그레이드하는 과정에서, Netty 기반 애플리케이션에서 CoreDNS 도메인 리졸빙 실패 현상이 발생함

- 업그레이드 중 CoreDNS Pod가 재시작되었고, 이후 Netty 애플리케이션에서 CoreDNS IP가 변경된 이후에도 기존 IP로 계속 DNS 요청을 전송
- 그로 인해 도메인 질의가 실패하며, Pod 재시작 전까지 DNS 장애가 지속
- 일반적인 Pod에서는 동일 현상이 재현되지 않았고, Netty 애플리케이션에서만 발생

문제점

- kube-proxy 1.29.7 버전의 known issue
  - 동일한 Source IP/Port에서 반복적으로 DNS Query가 발생할 경우, kube-proxy의 conntrack table에 기존 CoreDNS IP가 유지됨
  - CoreDNS Pod가 재시작되어 IP가 변경되어도 conntrack 갱신이 이루어지지 않아 이전 IP로 패킷이 전송됨
- Netty 특성
  - Netty는 동일 소켓(Channel)을 장기간 유지하며 고정된 Source IP/Port로 DNS 요청을 전송하기 때문에 해당 문제에 취약

개선 사항

- kube-proxy 업그레이드
  - kube-proxy를 1.29.9 이상 버전으로 업그레이드하여 conntrack 관련 이슈를 해결
- CoreDNS Lameduck 설정 조정
  - CoreDNS 재시작 시, lameduck 기간을 충분히 확보해 연결 종료 전 DNS 요청 실패가 발생하지 않도록 구성
- Netty 버전 패치
  - 동일 소켓 재사용 문제를 개선한 Netty 4.2 이상 버전으로 업그레이드하여 DNS 리졸빙 안정성 확보

적용 결과

- kube-proxy 및 Netty 버전 업그레이드를 통해 DNS 리졸빙 실패 재발 방지
- CoreDNS 재시작 및 네트워크 이벤트 발생 시에도 DNS 질의 실패 없이 안정적인 서비스 운영 가능

4. ArgoCD CMP를 통한 Helmfile 도구 활용

이슈 사항

Helmfile 도구를 통해 Kubernetes Manifest를 관리하고 있음

- 로컬 개발 환경에서는 Helmfile을 통해 환경별 변경 사항을 손쉽게 확인하고 관리할 수 있었지만, ArgoCD에서는 기본적으로 Helmfile을 지원하지 않음
- 따라서 ArgoCD를 통한 형상 관리를 위해서는 별도의 Kustomize나 Helm 설정을 추가로 구성해야 하였고, 이로 인해 로컬 개발 환경과 ArgoCD 운영 환경의 코드가 분리되는 문제가 발생함
- 변경 사항 발생 시, 두 환경의 코드를 모두 수정해야 하는 번거로움과 관리 포인트 증가라는 비효율이 존재했음

개선 사항

이를 해결하기 위해 ArgoCD의 CMP(Config Management Plugins) 기능을 도입

- ArgoCD의 Repo-server에 CMP 컨테이너를 사이드카로 구성
- Helmfile을 CMP 플러그인으로 활용하여, ArgoCD에서도 Helmfile로 Kubernetes Manifest를 렌더링 가능하도록 설정
- 로컬 환경에서 사용하던 Helmfile 설정을 그대로 ArgoCD에도 적용, 개발 및 운영 환경 간 코드 일관성 확보

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ConfigManagementPlugin
metadata:
  name: helmfile-plugin
spec:
  version: v1.0
  generate:
    command:
      - bash
      - "-c"
      - |
        if [[ -v ENV_NAME ]]; then
          if [[ -v ARGOCD_ENV_HELMFILE_CONFIG ]]; then
            helmfile -n "$ARGOCD_APP_NAMESPACE" -e $ENV_NAME -f $ARGOCD_ENV_HELMFILE_CONFIG template --include-crds -q
          else
            helmfile -n "$ARGOCD_APP_NAMESPACE" -e $ENV_NAME template --include-crds -q
          fi
        elif [[ -v ARGOCD_ENV_ENV_NAME ]]; then
          helmfile -n "$ARGOCD_APP_NAMESPACE" -e "$ARGOCD_ENV_ENV_NAME" template --include-crds -q
        else
          helmfile -n "$ARGOCD_APP_NAMESPACE" template --include-crds -q
        fi
  discover:
    find:
      command:
      - sh
      - "-c"
      - |
        find $ARGOCD_APP_SOURCE_PATH -name helmfile.yaml
```

적용 결과

- 로컬 개발 환경과 ArgoCD 운영 환경을 동일한 Helmfile 기반 코드로 통합 관리
- 수정 포인트 일원화로 관리 효율성 및 코드 일관성 향상
- 신규 서비스 배포나 환경별 구성 변경 시, 개발과 운영 간 설정 충돌 최소화

5. ArgoCD 성능 개선

이슈 사항

ArgoCD에서 MonoRepo 기반의 Helmfile Manifest 저장소를 관리하는 환경에서, 다음과 같은 이유로 ArgoCD의 성능 저하가 발생함

- MonoRepo 형태로 운영 중인 Manifest 저장소를 HEAD 브랜치 기준으로 전체 애플리케이션 관리
- 특정 애플리케이션 변경에도 모든 애플리케이션의 Manifest가 재생성
- Manifest 저장소 용량 증가로 인해 repo-server의 불필요한 리소스 소모

문제점

- Manifest 변경 감지 방식의 한계
  - ArgoCD는 Manifest 3분 주기로 저장소의 변경사항을 Polling 하며, Commit SHA 값을 캐시 키로 사용
  - HEAD 브랜치에 변경 발생 시 모든 애플리케이션의 캐시가 갱신되어 불필요한 Manifest 재생성 발생
- MonoRepo의 저장소 크기 문제
  - repo-server가 Manifest 저장소를 Clone 후, tar 압축해 CMP 컨테이너로 복사
  - tar 파일 내 `.git` 디렉터리가 포함되어, 전체 용량의 약 75%를 차지
  - 매번 압축, 복사, 삭제 과정을 반복하여 CPU 및 I/O 소모 증가
- Helm Charts 중복 다운로드 문제
  - CMP 컨테이너에서 helmfile 실행 시, 매번 Helm Charts를 새로 다운로드해 불필요한 네트워크, I/O 사용 발생

개선 사항

- Manifest 변경 감지 최적화
  - ArgoCD ApplicationSet에 `manifest-generate-paths` annotation 추가
  - GitHub Webhook을 통해 특정 애플리케이션의 변경 사항만 캐시를 갱신하도록 구성 (불필요한 전체 Manifest 재생성 방지)
- repo-server tar 용량 최적화
  - repo-server 실행 시, tar 압축 시 `.git` 디렉터리 제외 옵션 추가

```
repoServer:
  extraArgs:
  - --plugin-tar-exclude=.git/*
```

- Helm Charts 다운로드 최적화
  - CMP 컨테이너의 helmfile 실행 시, Event Hook을 통해 Helm Charts가 이미 존재하면 재사용

적용 결과

- CPU 사용량 대폭 감소
  - 평균 CPU 사용률 12 Core → 0.4 Core로 감소
- Manifest 재생성 횟수 감소
  - 주간 평균 7,000회 → 1,000회로 감소, Manifest 처리 속도 개선

6. Istio Ingress Gateway 도입으로 ELB 통합 및 비용 절감

이슈 사항

기존 EC2 레거시 환경에서는 하나의 API 당 External, Internal ELB를 개별적으로 구성해 운영하였음

- 서비스마다 ELB가 별도 생성되어, API 수 증가에 따라 ELB 수도 비례해 증가

EKS 환경으로 마이그레이션된 이후에도, 보안 그룹 성격에 따라 ELB를 각각 생성

- 이로 인해 ELB Target Group(Ingress Annotation)에 보안 그룹 설정이 중복 적용될 경우, 보안 그룹이 누락되어 서비스 장애로 이어질 가능성 존재 (장애 포인트 발생)

2024년 2월 1일부터 AWS에서 모든 퍼블릭 IP에 대한 과금 정책이 적용

- ELB 수가 많을수록 퍼블릭 IP 사용 비용이 지속적으로 증가하는 구조

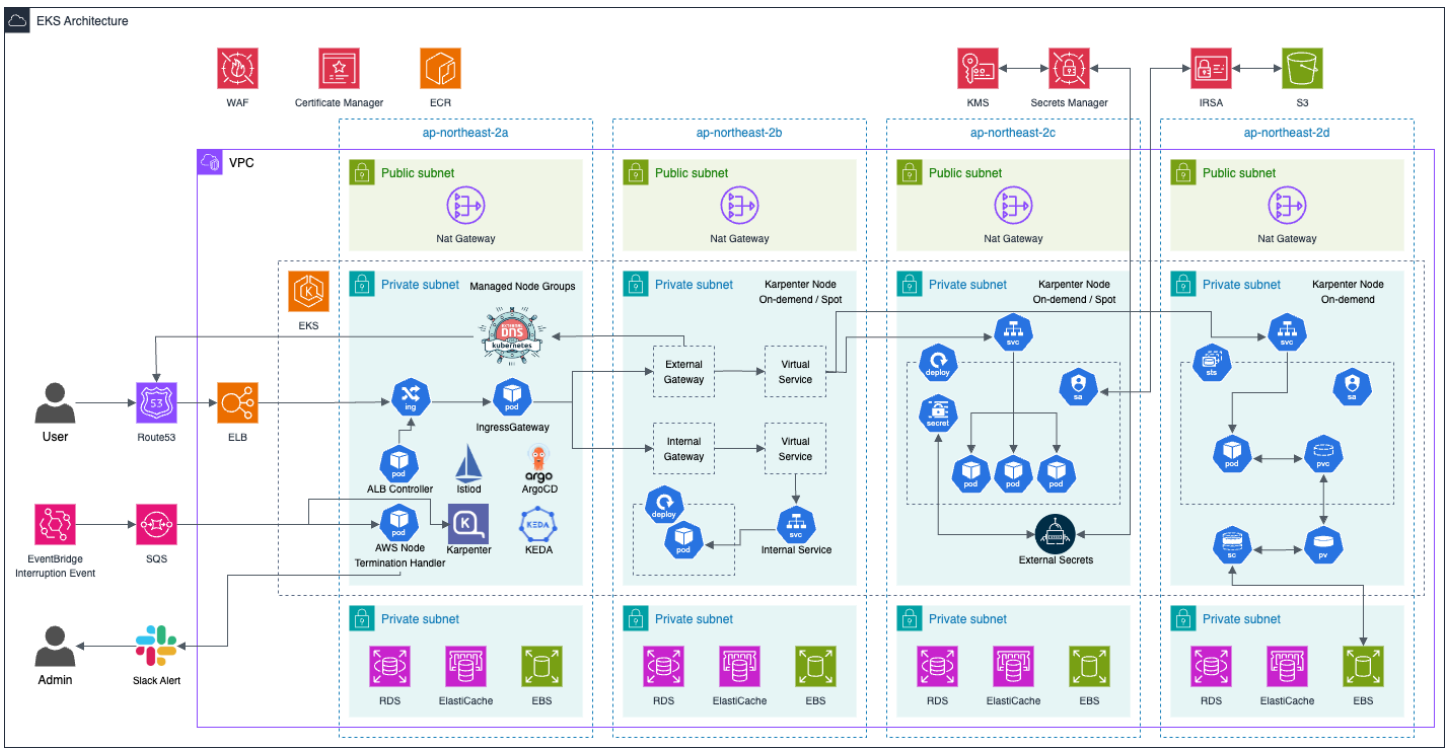
개선 사항

- 보안 그룹 기반으로 ELB를 다수 구성하는 방식 대신, Istio Ingress Gateway를 도입하여 ELB를 통합 관리
- 서비스 성격(External, Internal 등)에 따라 Istio Gateway를 통해 Ingress 구조를 표준화
- 기존 보안 그룹 설정은 AuthorizationPolicy를 활용해 IP 정책으로 관리
  - 보안 그룹 중복 설정에 따른 장애 포인트 제거

적용 결과

- 500개 이상의 ELB를 약 50개(클러스터 당 5개 이내)로 통합 운영
  - 퍼블릭 IP 비용을 월 1,000 USD → 약 70 USD로 절감
- AuthorizationPolicy를 통해 보안 정책을 일원화
  - ELB Target Group 설정 중복 문제로 인한 장애 포인트 해소
- 인프라 비용 최적화 및 운영 복잡성 감소, 서비스 확장성 및 안정성 향상

- Istio를 적용하여 구축한 EKS 아키텍처



7. GPU 노드 효율 운영을 위한 이벤트 기반 스케일링 아키텍처 설계

이슈 사항

EKS 환경에서 신규 서비스가 GPU 리소스를 활용해야 하는 상황

- GPU 노드는 일반 인스턴스에 비해 비용이 높기 때문에, 온디맨드 방식으로 상시 운영할 경우 불필요한 리소스 낭비와 비용 증가가 우려됨
- 서비스 특성상 GPU가 일정 시점에만 필요한 구조였기 때문에, 온디맨드 방식으로 자원을 운영할 경우 비효율성 증가 가능성이 존재함

실행 방안

- GPU 노드를 필요할 때만 유연하게 운영할 수 있도록, 이벤트 기반 스케일링 아키텍처 설계
- KEDA 도입
  - 큐에 작업 메시지가 쌓이면 KEDA가 자동으로 감지해 GPU 노드를 스케일아웃
  - 작업 완료 후, 노드는 자동 회수되어 유휴 리소스 발생 방지
- Karpenter 활용
  - GPU 노드의 프로비저닝과 회수 과정을 자동화하여 불필요한 자원 소모 최소화
  - 스팟 인스턴스를 우선 사용해 비용 최적화
- 스팟 인스턴스 회수 위험 대응
  - 작업 중 인스턴스가 회수될 경우를 대비해, DLQ(Dead Letter Queue)를 적용
  - 작업 실패 시 메시지 재처리로 작업 안정성 확보

적용 결과

- GPU 노드를 작업 상황에 맞춰 동적으로 운영하여 고비용 리소스를 최소한의 비용으로 효율적으로 활용
- 스팟 인스턴스를 회수하여도 DLQ를 통해 작업 안정성 확보하여 서비스 신뢰성 유지
- 온디맨드 노드 대신 이벤트 기반 스케일링 구조를 도입함으로써, 운영 비용 절감과 자원 활용 효율성 극대화

8. Terraform을 활용한 AWS 리소스 관리

이슈 사항

EKS 기반으로 신규 서비스를 구성할 때마다, 다음과 같은 AWS 리소스를 반복적으로 생성해야 하는 비효율이 존재

- ECR Repository
- IRSA (IAM Role for Service Account)
- Security Group (Pod 보안 그룹)

서비스마다 동일한 리소스 설정을 중복 작성해야 했고, 관리 일관성 유지가 어려운 문제가 발생함

개선 사항

- 커뮤니티에서 제공하는 `terraform-aws-modules` 를 활용하여 ECR, IRSA, Security Group 등 서비스 구성에 필요한 AWS 리소스를 모듈화하여 재사용 가능하도록 구성
- 서비스 디렉터리 하위에 환경별 디렉터리(dev, stage, qa, prod)를 구성하여 각 환경에 필요한 리소스를 쉽게 설정하고 관리할 수 있도록 구조화
- Terraform State 파일 관리 최적화
  - 코드 내 경로와 S3의 Remote State 경로를 동일하게 맞춰, State 위치 식별과 관리 편의성 확보
  - 환경별 State 파일 분리로 충돌 방지 및 작업 복잡성 최소화
- Atlantis 도입
  - GitHub과 연동해 인프라 변경 사항을 PR 기반으로 적용
  - PR 리뷰, 승인, 롤백 등 변경 이력을 체계적으로 관리 가능하도록 구성

적용 결과

- 모듈화된 Terraform 구성을 통해 AWS 리소스 재사용성 향상 및 일관된 리소스 관리 체계 확보
- 환경별 State 분리로 작업 충돌 방지, 상태 관리 복잡성 해소
- Atlantis 기반 PR 프로세스 도입으로 변경 이력 가시성 및 배포 효율성 개선
- 인프라 운영 전반에 변경 관리 안정성, 생산성, 속도 모두 향상

9. Teleport를 활용한 EKS 접근 권한 관리

이슈 사항

EC2 환경에서 EKS 환경으로 서비스 마이그레이션을 진행하면서, EKS 클러스터 접근 권한 관리 체계 수립이 필요해짐
기존에는 IAM 역할과 `aws-auth` ConfigMap을 통해 RBAC을 구성했으나, 다음과 같은 한계가 존재

- IAM Role과 ConfigMap 변경 시 환경별 설정 관리가 복잡하고 번거로움
- 인증 및 접근 로그가 여러 위치에 분산되어 내부 감사 대응 및 이력 추적이 비효율적

의사 결정 사항

아래와 같은 이유로 EKS 접근 권한 관리를 Teleport 기반으로 전환하기로 결정

- 내부회계 관리 제도(ITGC) 감사 대응 요구
  - Jenkins, ArgoCD, Teleport 등 3rd-party 서비스의 인증 방식을 GitHub SSO로 통일하여 관리 일원화
  - 모든 접근 로그가 중앙화되어 추적 가능성 확보
- 세션 녹화 기능 제공
  - 개발자가 `kubectl` 명령어로 EKS 클러스터에 접근 시, Teleport를 통해 세션 자동 녹화
  - 감사 및 보안 요구사항 충족

- ISMS 인증 심사 및 감사 대응을 위한 권한 통합 관리 필요
  - EKS 클러스터 접근 제어를 GitHub 인증 기반으로 통합
  - 권한 관리 일원화로 관리 포인트 최소화
- 보안 환경 분리 구성
  - 개발 환경 EKS 클러스터는 보안 개발 VPC에 존재하는 Teleport와 연결
  - 운영 환경 EKS 클러스터는 보안 운영 VPC에 존재하는 Teleport와 연결하여 VPC 분리

적용 결과

- 개발자는 GitHub 인증을 통해 Teleport에 로그인한 뒤, 자신이 속한 GitHub 팀에 따라 권한이 있는 EKS 클러스터 및 네임스페이스에만 접근 가능하도록 구성
- 접근 세션 자동 녹화 및 인증 로그 관리가 중앙화되어, ITGC 및 ISMS 등 보안 감사 대응 효율성 향상
- 기존 IAM + ConfigMap 방식 대비 권한 관리 복잡성 감소 및 보안 통제 강화

10. OpenStack 네트워크 구조 개선 작업

이슈 사항 1: 네트워크 병목 현상

기존 OpenStack 환경에서는 컴퓨팅 노드가 외부와 통신할 때, 모든 트래픽이 컨트롤러 노드를 경유해야 하는 구조였음

- 초기에는 컴퓨팅 노드 수가 적어 문제가 없었지만, 인프라 확장으로 컴퓨팅 노드가 200대 이상 증설되면서 컨트롤러 노드에 트래픽이 집중되어 네트워크 병목 현상이 발생

개선 사항 1: OpenStack DVR(Distributed Virtual Routing) 기능 도입

- 기존에 컨트롤러 노드에 집중되어 있던 가상 라우터를 모든 컴퓨팅 노드에 분산 배치
- 컴퓨팅 노드가 외부 통신 시 컨트롤러를 거치지 않고 직접 라우팅 가능
- 트래픽 집중 해소 및 네트워크 확장성 확보

이슈 사항 2: VM 통신 불안정 문제

DVR 기능 도입 이후, 30분 주기로 VM 간 통신 및 VM과 컴퓨팅 노드 간 통신이 불안정해지는 현상 발생

- 일정 시간이 지나면 VM 간 패킷 손실 및 연결 끊김 현상이 지속적으로 발생

개선 사항 2: OpenStack Open vSwitch 버그 패치

- 문제 발생 시점에 네트워크 패킷 덤프 분석 진행, 특정 구간에서 MAC Address 유실 현상 확인
- 원인 분석 결과, OpenStack Open vSwitch 모듈의 버그로 인한 문제로 판단
- 커뮤니티에 이슈를 공유하고, 공식 패치 적용 및 OVS 모듈 업데이트를 통해 문제 해결

적용 결과

- DVR 기능 도입으로 트래픽 분산, 컨트롤러 노드에 집중되던 네트워크 병목 문제 해소
- OVS 버그 패치 적용으로 VM 간 통신 단절 문제 해결, 외부 연결 안정성 확보
- 네트워크 구조 전반을 개선하여 확장성과 안정성을 모두 갖춘 OpenStack 환경 구축